

BIQLY

Teknik Mimari & Best-Practice Analiz Dokümanı

Doğal dilden SQL üreten (Text-to-SQL) yapay zekâ destekli iş zekâsı platformu

Kapsam: Go backend · React/TypeScript frontend · CI/CD pipeline'ları · Kubernetes dağıtımı · kalite denetimi

Sürüm 1.0 · Haziran 2026 · Hibrit (yönetici özeti + teknik derinlik)

Hazırlayan: Mimari Analiz · go.mod: github.com/biqly/biqly (Go 1.26)

İçindekiler

1. Yönetici Özeti	4
2. Sistem Genel Bakışı	5
2.1 Teknoloji Yığını (Özet)	5
3. Mimari Genel Bakış	6
3.1 Servisler / Çalıştırılabilir Binary'ler	6
3.2 Servisler Arası İletişim	7
4. Backend Analizi (Go)	8
4.1 internal/ Paketleri	8
4.2 Teknik Notlar	8
5. Frontend Analizi (React / TypeScript)	9
5.1 Uygulama Mimarisi ve Yönlendirme	9
5.2 Bileşen Envanteri (özet)	9
5.3 API Katmanı, Durum ve i18n	9
6. Temel Akışlar (Pipeline'lar)	10
6.1 Text-to-SQL Akışı	10
6.2 Kimlik Doğrulama (Auth) Akışı	11
6.3 Semantik Katman ve Derleme Pipeline'ı	11
7. HTTP Katmanı ve Veri Katmanı	12
7.1 HTTP Middleware Yığını	12
7.2 Veri Katmanı	12
8. CI/CD Pipeline'ları ve Dağıtım	13
8.1 GitHub Actions İş Akışları	13
8.2 Docker İmajları	13
8.3 Kubernetes Runtime Topolojisi	14
9. Best-Practice Değerlendirmesi	15
9.1 Güvenlik	15
9.2 Test	16
9.3 Kod Kalitesi & Mimari	16

9.4 Gözlemlenebilirlik	17
9.5 Performans	17
9.6 AI / LLM Mühendisliđi	18
9.7 DevX / Sürdürülebilirlik	18
10. Sonuç ve Yol Haritası	19

1. Yönetici Özeti

Biqly; kullanıcıların doğal dilde (Türkçe/İngilizce) sorduğu soruları güvenli, doğrulanmış SQL sorgularına çevirip çalıştıran, yapay zekâ destekli bir **iş zekâsı (BI) ve veri analitiği platformudur**. Sistem; Go ile yazılmış bir backend (modüler monolit / mikroservis hibridi), React 19 + TypeScript ile yazılmış tek sayfa uygulaması (SPA) frontend ve Helm + ArgoCD ile yönetilen GitOps tabanlı bir Kubernetes dağıtımından oluşur.

Bu doküman iki katmanlıdır: önce karar vericilere yönelik üst seviye değerlendirme, ardından geliştiriciler ve mimarlar için teknik derinlik sunar. Her bölümde sistemin **nasıl çalıştığı** anlatıldıktan sonra **güçlü yönler, riskler ve somut iyileştirme önerileri** verilmiştir.

Öne çıkan tespitler

- **AI/Text-to-SQL motoru platformun en güçlü ve farklılaştırıcı bileşenidir.** LLM doğrudan SQL değil, ara bir *LogicalQuery* üretir; bu sorgu semantik modele karşı doğrulanır ve ancak ondan sonra lehçeye özel SQL'e derlenir. EXPLAIN ile kuru-çalıştırma (dry-run), kendi kendini onaran yeniden deneme döngüsü, öz-tutarlılık oylaması ve güven skorlaması ile birlikte bu, tipik “prompt-and-pray” yaklaşımının çok üzerindedir.
- **Güvenlik katmanlı ve özenlidir:** RS256 JWT, zamanlama-güvenli CSRF, mutlak + boşta kalma oturum süreleri, AES-256-GCM ile şifrelenmiş sağlayıcı anahtarları, parametrelili SQL ve beyaz-liste tabanlı sorgu doğrulaması.
- **Mühendislik disiplini yüksektir:** temiz `cmd/internal/pkg` ayrımı, modern Go 1.26 hata deyimleri (`143× errors.Is`), ~55 linter, ESLint+knip+Prettier kapısı ve zorunlu pre-commit denetimleri.
- **Üç odaklı boşluk:** (1) dağıtık izleme (tracing) altyapı olarak kurulu ama kodda enstrümente edilmemiş; (2) lehçeye özel veri kaynağı sürücülerinde test kapsamı ince; (3) `AIConfig` tanrı-nesnesi ve devasa orkestratör fonksiyonu yerel bakım borcu oluşturuyor. Hiçbiri mimari kusur değil; kademeli olarak giderilebilir.

Genel olgunluk değerlendirmesi

Biqly, bir prototip değil; “**odaklı boşluklarla birlikte üretime hazır**” olgunlukta, geç-aşama bir ürün kod tabanıdır. Aşağıdaki skor kartı, yedi mühendislik boyutundaki değerlendirmeyi 5 üzerinden özetler:



Şekil 1 — Mühendislik olgunluğu skor kartı (5 üzerinden, kanıta dayalı tahmini değerlendirme).

2. Sistem Genel Bakışı

Biqly'nin temel iş akışı şudur: bir kullanıcı bir veri kaynağı seçer ve doğal dilde bir soru sorar (örn. “geçen çeyrekte en çok satan 10 ürün”). Platform; ilgili tabloları otomatik olarak yönlendirir (table routing), bir semantik model bağlamı kurar, LLM'den yapısal bir mantıksal sorgu üretmesini ister, bu sorguyu doğrular ve lehçeye uygun SQL'e derleyip çalıştırır; sonuç ve üretilen SQL kullanıcıya geri döner.

Platform aynı zamanda; **semantik modelleme** (sürükle-bırak tuval), **veri kataloğu / metadata yönetimi**, **panolar (dashboards)**, **rol tabanlı erişim (RBAC)**, **PII tespiti ve maskeleye**, **denetim kayıtları** ve **AI değerlendirme (eval) koşum hattı** gibi kurumsal yetenekler sunar.

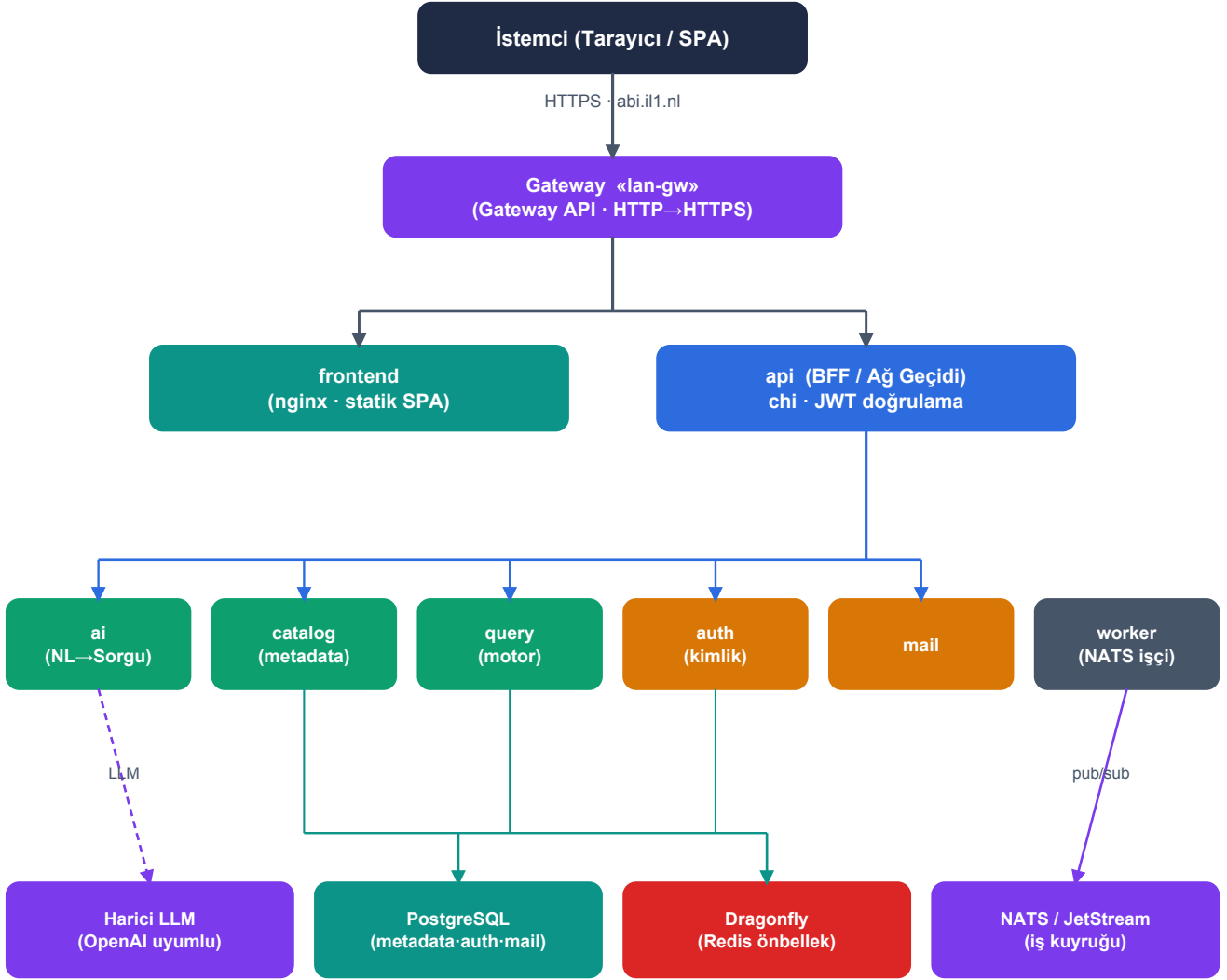
2.1 Teknoloji Yığını (Özet)

Katman	Teknoloji / Kütüphane	Not
Backend dili	Go 1.26.4	Modül: github.com/biqly/biqly
HTTP yönlendirme	go-chi/v5, go-chi/cors	Tüm servislerde ortak
Veritabanı	PostgreSQL (pgx/v5)	Bitnami subchart · 3 mantıksal DB
Harici veri kaynakları	MySQL · SQL Server · ClickHouse	Sürücü + lehçe soyutlaması
Mesajlaşma	NATS / JetStream	Asenkron AI iş kuyruğu
Önbellek	Dragonfly (Redis uyumlu)	Rate-limit, AI yanıt önbelleği, RBAC
Kimlik/kripto	JWT RS256 · WebAuthn · LDAP · OAuth	golang-jwt, go-webauthn
JSON (sıcak yol)	bytedance/sonic · json-iterator	Performans için
Frontend	React 19 · TypeScript 5.7 · Vite	Vanilla CSS + BEM, i18n
Grafik	Recharts 3.8	Pano ve görselleştirme
Dağıtım	Helm umbrella + ArgoCD	GitOps · ghcr.io/biqly/*
Gözlemlenebilirlik	slog · Prometheus · (OTEL/Jaeger kurulu)	Tracing kodda eksik (bkz. §9)

*Mimari notu: Backend bir **modüler monolittir ve mikroservislere bölünebilir**. Aynı `internal/` paketleri farklı binary'lere bağlanır. Bir isteğin süreç-içi mi işleneceği yoksa eş servise mi proxy'leneceği saf bir konfigürasyon kararıdır (`BI_*_SERVICE_URL` ortam değişkeninin varlığı).*

3. Mimari Genel Bakış

Sistem; bir **API/BFF (Backend-for-Frontend) ağ geçidi** ile onun arkasındaki eş servislerden oluşur. Ağ geçidi kimlik doğrulama ve yetkilendirmeyi sınırdan uygular; doğrulanmış istekleri ya süreç-içi işler ya da ilgili eş servise ters-proxy'ler. Servisler arası tüm çağrılar tipli HTTP+JSON istemcileri üzerinden ve yalnızca küme içinden yapılır.



düz çizgi: HTTP/JSON · - - - kesik: dış LLM çağrısı (HTTPS) · pub/sub: NATS · tüm servisler ayrıca auth'tan JWT açık-anahtarını çeker

Şekil 2 — Mantıksal servis topolojisi ve servisler arası çağrı grafiği.

3.1 Servisler / Çalıştırılabilir Binary'ler

Binary	Sorumluluk	Protokol
api	Ana API sunucusu / BFF. Tüm bağımlılıkları bağlar; catalog/query/AI'yi süreç-içi sunar veya eş servislere proxy'ler. AI iş tüketicisini başlatır.	HTTP (chi) · pprof
auth	Bağımsız kimlik servisi. JWT üretimi, oturum, RBAC, MFA, LDAP, OAuth, magic-link, davet. JWT açık anahtarını yayınlar.	HTTP · /metrics
ai	Bağımsız AI servisi. NL→sorgu hattı + iş tüketicisi. Ağa güvenir; yetkilendirmeyi proxy uygular.	HTTP

Binary	Sorumluluk	Protokol
catalog	Veri kaynakları, metadata, semantik modeller, panolar, izinler, drift.	HTTP
query	Sorgu motoru: mantıksal sorguyu derler / çalıştırır / EXPLAIN ile doğrular.	HTTP
worker	Başsız arka plan işçisi. NATS'tan AI işlerini tüketir.	NATS tüketici
mail	İşlemsel e-posta işçisi. SMTP gönderimi, yeniden deneme, blok listesi.	HTTP
migrate / auth-migrate / mail-migrate	İlgili veritabanlarının şema göçleri (CLI).	CLI
export-sft	Gemma ince-ayarı için SFT veri kümesi (JSONL) dışa aktarımı.	CLI

3.2 Servisler Arası İletişim

- **api (ağ geçidi)** → /api/datasources, /semantic, /metadata, /dashboards, /permissions → **catalog**; /api/query/* → **query**; /api/ai/* → **ai**.
- **ai servisi** → catalogclient (model/metadata/few-shot/sözlük), queryclient (compile/run/dry-run), geçmiş catalog'a yazar.
- **Tüm servisler** → auth /internal/auth/public-key (JWT RSA açık anahtarı), /check-permission, /check-datasource-access.
- **auth** → mail (işlemsel e-posta gönderimi).
- **İç yüzey** (/internal/*) genel ağ geçidinden asla eşleşmez; X-Internal-Token ile korunur.

4. Backend Analizi (Go)

Backend; 73 paket, ~593 dosya ve ~5.458 sembolden oluşur. Kod tabanı; sorumlulukları net biçimde ayıran `cmd/` (giriş noktaları), `internal/` (uygulama) ve `pkg/` (paylaşılan SDK) üçlüsüne dayanır.

4.1 internal/ Paketleri

Paket	Sorumluluk
ai	Metni sorguya çeviren orkestratör (<code>Service.ProcessQuestion</code>). Alt paketler: <code>prompt</code> , <code>provider</code> , <code>routing</code> (tablo yönlendirme), <code>eval</code> , <code>ambiguity</code> , <code>abtest</code> , <code>embedder</code> , <code>response_cache</code> .
app	Bağımlılık enjeksiyonu kökü. <code>NewDependencies/NewAIDependencies/...</code> ; süreç-içi mi yoksa istemci-adaptör mü kararını verir.
auth	Kimlik çekirdeği: <code>Service</code> , <code>JWTManager</code> (RS256), <code>SessionManager</code> , <code>CSRF</code> , <code>RateLimiter</code> ; <code>rbac</code> , <code>mfa</code> (TOTP+WebAuthn), <code>ldap</code> , <code>oauth</code> , <code>workspace</code> .
core	Servis seviyesi sorgu orkestrasyonu (<code>Compile/Run/DryRun</code>) ve tipli <code>ServiceError</code> .
query	Mantıksal sorgu → SQL: <code>Compiler</code> , <code>expr_compiler</code> (AST→SQL + PII maskeleyme), <code>Validator</code> , <code>ComputeFingerprint</code> .
semantic	Semantik model alanı: <code>Dimension/Metric/Join</code> , <code>CompositeModel</code> , <code>metric_graph</code> (bağımlılık grafiği + döngü tespiti), <code>drift</code> .
datasource	DB sürücü soyutlaması: <code>Driver</code> arayüzü, <code>Registry</code> , <code>postgres/mysql/sqlserver/clickhouse</code> , <code>pool_cache</code> .
dialect	Motora özel SQL üretimi (<code>CoreDialect/SampleDialect</code>), tanımlayıcı tırnaklama.
security	Encryption (AES), DSN redaksiyonu, güvenilir-proxy IP, <code>RowFilter</code> (satır seviyesi güvenlik), read-only SQL koruması, <code>pii</code> .
metadata	Metadata Postgres deposu: <code>datasources/tables/columns/relations</code> , <code>few-shot</code> , sözlük, <code>ai_query_history</code> , <code>ai_jobs</code> .
queue	AI iş kuyruğu soyutlaması: <code>NATSQueue</code> (<code>JetStream</code> , <code>dead-letter</code>) + <code>LocalAIJobQueue</code> (geliştirme).
http	Yönlendirme + handler'lar + middleware yığını (§7).
audit	Asenkron sorgu/kimlik denetim kaydı (tamponlu <code>DBWriter</code>).
dbmigrate / config / i18n / mail / dashboard / platform / errmsg / semanticgen	Sırasıyla: göç koşturucusu, tipli konfig (+GOMEMLIMIT), TR/EN i18n, mail iç bileşenleri, pano deposu, <code>db/redis/observability</code> , kararlı hata kodları, <code>metadata</code> 'dan otomatik model üretimi.

4.2 Teknik Notlar

- LLM SDK bağımlılığı yok:** AI sağlayıcı istemcisi `internal/ai/provider` içinde elle yazılmış, OpenAI-uyumlu bir HTTP istemcisidir; sağlayıcı/model DB'den yönetilir ve sıcak-yeniden-yüklenir (`ProviderStore`).
- Üç mantıksal veritabanı:** `metadata`, `auth` ve `mail` ayrı DSN'lerle izole edilir.
- Graceful shutdown** tüm HTTP sunucularında (`SIGINT/SIGTERM`) ve yapılandırılmış `slog` ile loglama.

5. Frontend Analizi (React / TypeScript)

Frontend; AI destekli BI platformunu önyüzleyen tek sayfa bir React uygulamasıdır (~65 bin satır TS/TSX/CSS). Bilinçli olarak **bağımlılık-hafif** tasarlanmıştır: Redux/React-Query gibi global durum/veri-çekme kütüphaneleri yoktur — durum React Context + özel hook'lar, veri çekme elle yazılmış bir sarmalayıcı ile yapılır.

5.1 Uygulama Mimarisi ve Yönlendirme

`main.tsx` iç içe sağlayıcı (provider) ağacını kurar: `I18n` → Theme → Confirm → Toast → Shortcuts → `AIJobs` → Router → Auth. Yönlendirme **react-router-dom v7** ile yapılır; iki üst dal vardır: `GuestGuard` altındaki misafir rotaları (giriş, kayıt, şifre sıfırlama) ve `AuthGuard` altındaki uygulama kabuğu.

- **Kod bölme:** her rota `lazyWithPreload()` ile tembel yüklenir; kenar çubuğu fareyle üzerine gelince ilgili chunk'ı önceden ısıtır (preload).
- **Uygulama kabuğu:** daraltılabilir kenar çubuğu, `⌵` komut paleti, breadcrumb, workspace seçici, dil değiştirici, tema değiştirici ve rota başına `ErrorBoundary`.

5.2 Bileşen Envanteri (özet)

Alan	Başlıca Bileşenler
AI doğal dil sorgu / sohbet	AIQuery + aiQuery/ (ChatPanel, AssistantMessageCard, RoutingPanel/routingViz, FeedbackSection)
Semantik modelleme tuvali	Modeling + modeling/ (ModelingCanvas, useModelingCanvas, canvasMath) — sıfırdan yazılmış sürükleyici/yakınlaştır tuval
Sorgu oluşturucu	QueryBuilder + adım bileşenleri (Fields/Filter/Summarize/Sort/Having/Window/Cte)
Metadata / veri kataloğu	Metadata, Datasources, TableBrowser, ResultTable, AI describe modal'ları
Panolar & analitik	Dashboard, DashboardBuilder (Recharts), AIUsageDashboard
Değerlendirme (eval)	Evaluation + EvalRun/Regression/History sekmeleri
Kimlik	auth/ (SignIn, SignUp, ForgotPassword, OAuthCallback, PasswordStrengthMeter, AuthProvider/Guard)
Yönetim (admin)	Admin sekmeli kabuğu: kullanıcı/rol/RLS/alan-izni/PII/LDAP/AI sağlayıcı/AB-deney/drift panelleri

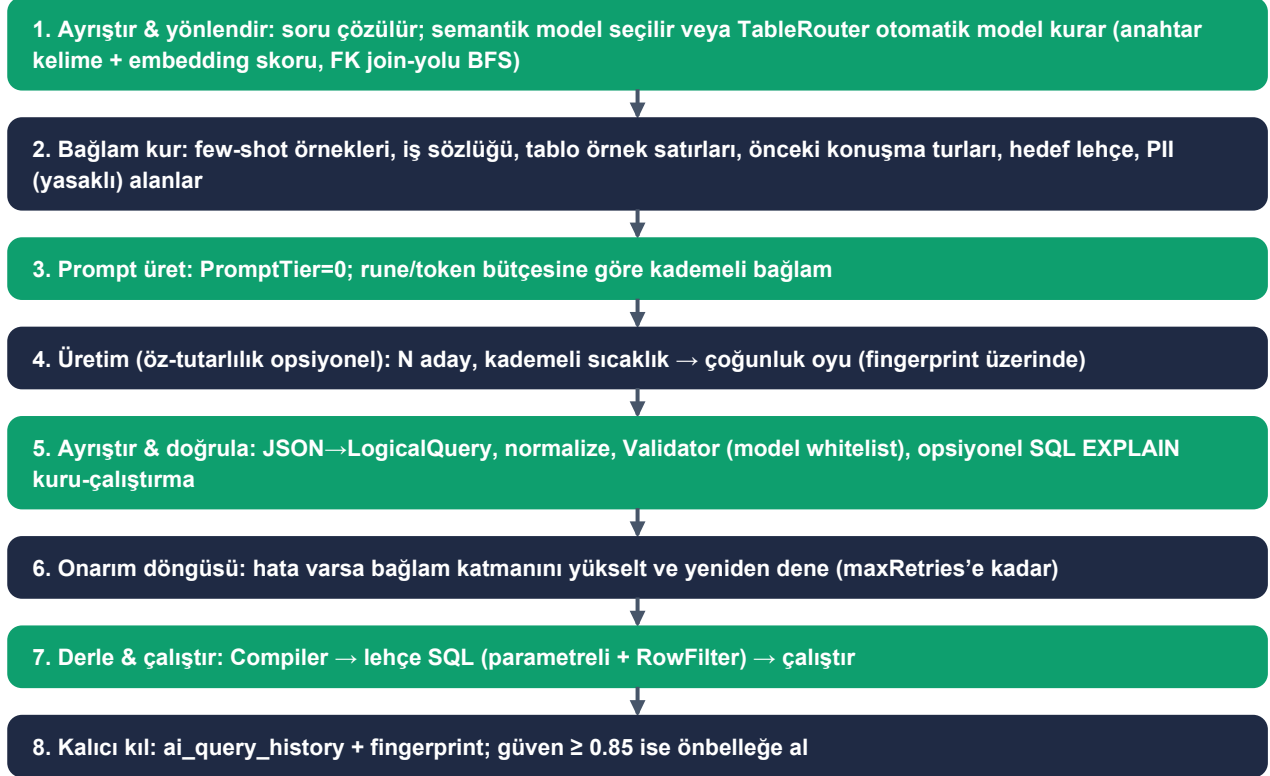
5.3 API Katmanı, Durum ve i18n

- **Fetch çekirdeği** (`apiClient.ts`): tipli `fetchJSON<T>`, `AbortController` zaman aşımı, otomatik `X-Locale` başlığı, Bearer token.
- **CSRF** (`csrf.ts`): tüm istekler `csrfFetch` üzerinden çift-gönderim deseniyle; güvenli metotlar token'ı yakalar, güvensiz metotlar `X-CSRF-Token` enjekte eder.
- **Durum & kimlik:** global store yok — Context + hook'lar (`AuthProvider`, `useApi`, `useAIJobs poller`); erişim token'ı yalnızca bellekte, refresh token `localStorage`'da, 14 dk'da bir sessiz yenileme.
- **i18n:** kütüphanesiz, **derleme-zamanı tip-güvenli** çeviri anahtarları; diller TR (varsayılan) ve EN; admin/auth sözlükleri tembel yüklenir.
- **Stil:** Vanilla CSS + BEM; `index.css` tasarım-sistemi çekirdeği (CSS değişkenleri, light/dark tema), özellik bazlı ~38 CSS dosyası.

6. Temel Akışlar (Pipeline'lar)

6.1 Text-to-SQL Akışı

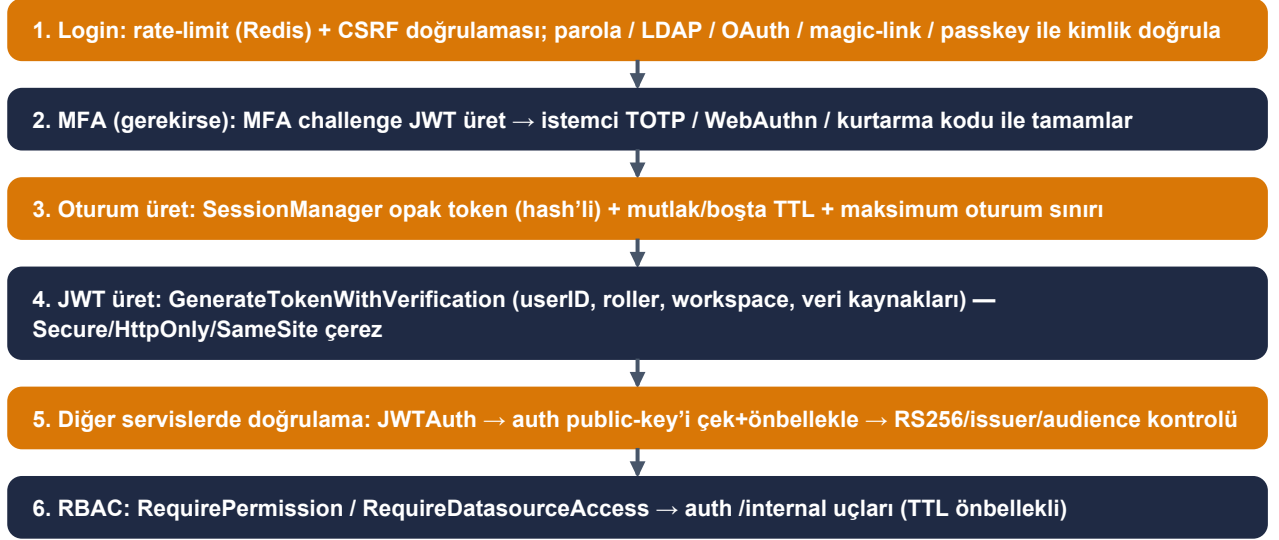
Platformun kalbi budur. Giriş: `POST /api/ai/query` (önizleme) veya `/api/ai/query/run`. Ağ geçidi `ai:query` iznini ve veri kaynağı erişimini uygular; sonra istek AI hattına girer. Aşağıdaki adımlar bir sıralı pipeline oluşturur:



Şekil 3 — Text-to-SQL kendi-kendini-onaran pipeline'ı. Güven skoru: $0.9 - 0.15 \cdot \text{doğrulama_hatası} - 0.10 \cdot \text{yeniden_deneme}$.

6.2 Kimlik Doğrulama (Auth) Akışı

Token modeli: **RS256 JWT erişim token'ları** (auth tarafından üretilir) + **opak, hash'lenmiş, DB tabanlı refresh oturumları**.



Şekil 4 — Kimlik doğrulama ve yetkilendirme akışı (RS256 JWT + opak DB oturumu + RBAC).

6.3 Semantik Katman ve Derleme Pipeline'ı

- **metric_graph.go:** kompozit modeller için metrik bağımlılık grafiği (MetricNode/Edge) kurulur; yayından/derlemeden önce DetectCircularDependencies ile döngü tespiti yapılır.
- **compiler.go:** LogicalQuery → CompiledQuery (SQL + args); join'ler join-grafiği üzerinden otomatik belirlenir; SELECT/WHERE/GROUP BY/HAVING/ORDER BY + pencere fonksiyonları kurulur. **Değerler asla string'e gömülmez (parametrelili).**
- **expr_compiler.go:** ExprNode AST'i lehçe SQL'e çevrilir; PIIMaskingConfig ile PII maskeleye uygulanır.
- **validator.go:** alan/dimension/metric varlığı, tarih-filtre tipleri, satır üst sınırı kontrolleri; hatalar kararlı errmsg kodları taşır — AI onarım döngüsü bu kodları okur.
- **fingerprint.go:** sorgu + semantik model sürümü kanonikleştirilip kararlı hash üretilir; önbellek, geçmiş tekilleştirme ve öz-tutarlılık oylamasında kullanılır.

7. HTTP Katmanı ve Veri Katmanı

7.1 HTTP Middleware Yığını

Yığın (monolit Router): RequestID → trace-context yayılımı → istek logu → ReallP → chi Logger → Recoverer → SecurityHeaders (HSTS + CSP) → Locale → CORS (açık izin listesi). Auth servisi ek olarak CSRF ve RateLimiter ekler.

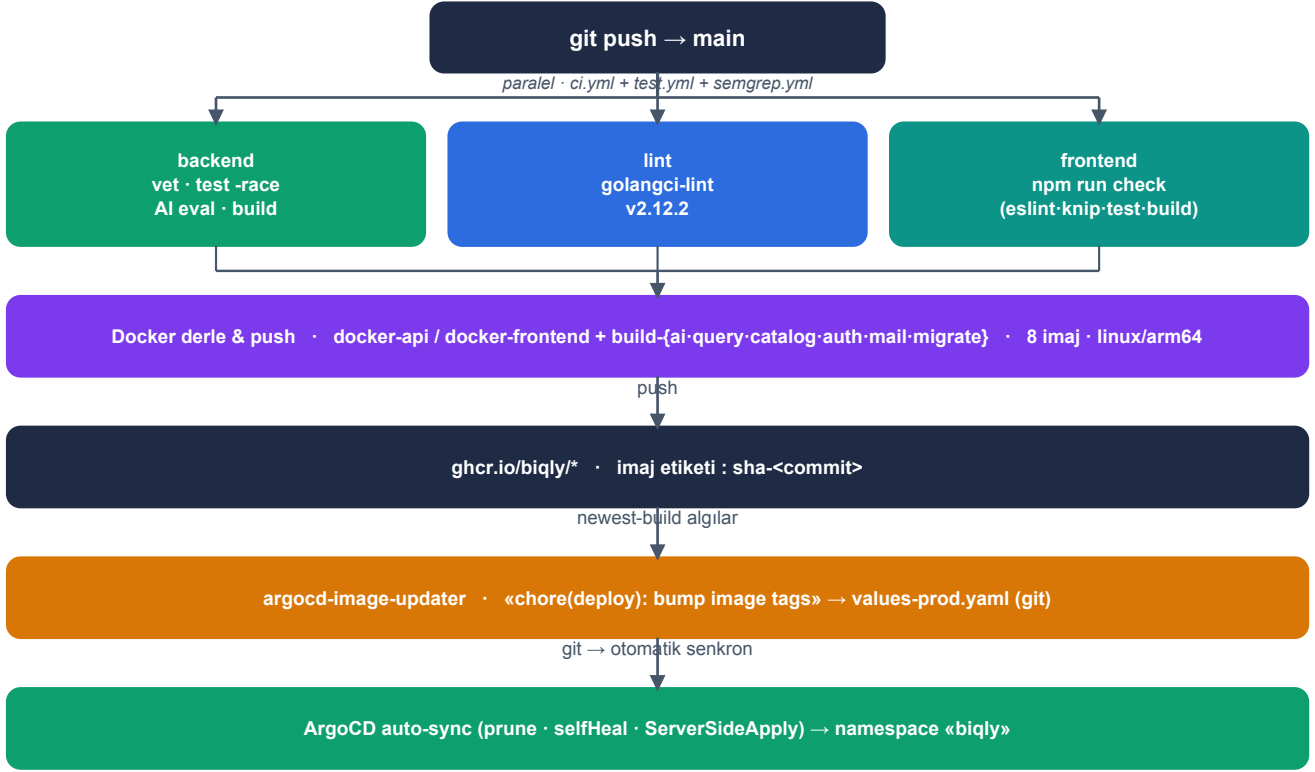
Rota grupları: `/health`, `/ready`, `/metrics` (Prometheus); JWT korumalı `/api/*` (catalog, query, ai grupları, her rotada izin + veri kaynağı erişim kontrolü); küme-içi `/internal/*` (X-Internal-Token + denetim). Bir `BI_*_SERVICE_URL` ayarlandığında ilgili `/api` grubu ters-proxy ile değiştirilir.

7.2 Veri Katmanı

Bileşen	Kullanım
PostgreSQL (pgx/v5)	Üç mantıksal DB: metadata, auth, mail. Harici kullanıcı veri kaynakları ayrıca MySQL/SQL Server/ClickHouse.
Göçler (dbmigrate)	Numaralı .sql dosyaları, schema_migrations + dirty-state tespiti; migrate/auth-migrate/mail-migrate binary'leri.
NATS / JetStream	Dayanıklı AI iş kuyruğu; yeniden teslim + dead-letter (maxDeliver). Uzun süren işler (batch describe, embedding).
Dragonfly / Redis	Rate-limit, AI yanıt önbelleği, RBAC veri-kaynağı erişim listeleri, oturum/OAuth durumu.
ai_jobs (metadata)	İş durumu/faz/ilerleme, çakışma tespiti, atomik TryMarkAIJobRunning (tek-sefer talep), bayat-iş toplama.

8. CI/CD Pipeline'ları ve Dağıtım

Monorepo; tek bir Kubernetes namespace'ine (biqly) Helm + ArgoCD GitOps ile dağıtılır. Konteyner kayıt defteri: `ghcr.io/biqly/*`. CI koşucu mimarisi ARM64'tür; imajlar `linux/arm64` olarak üretilir.



Şekil 5 — Commit'ten dağıtıma uçtan uca CI/CD ve GitOps pipeline'ı.

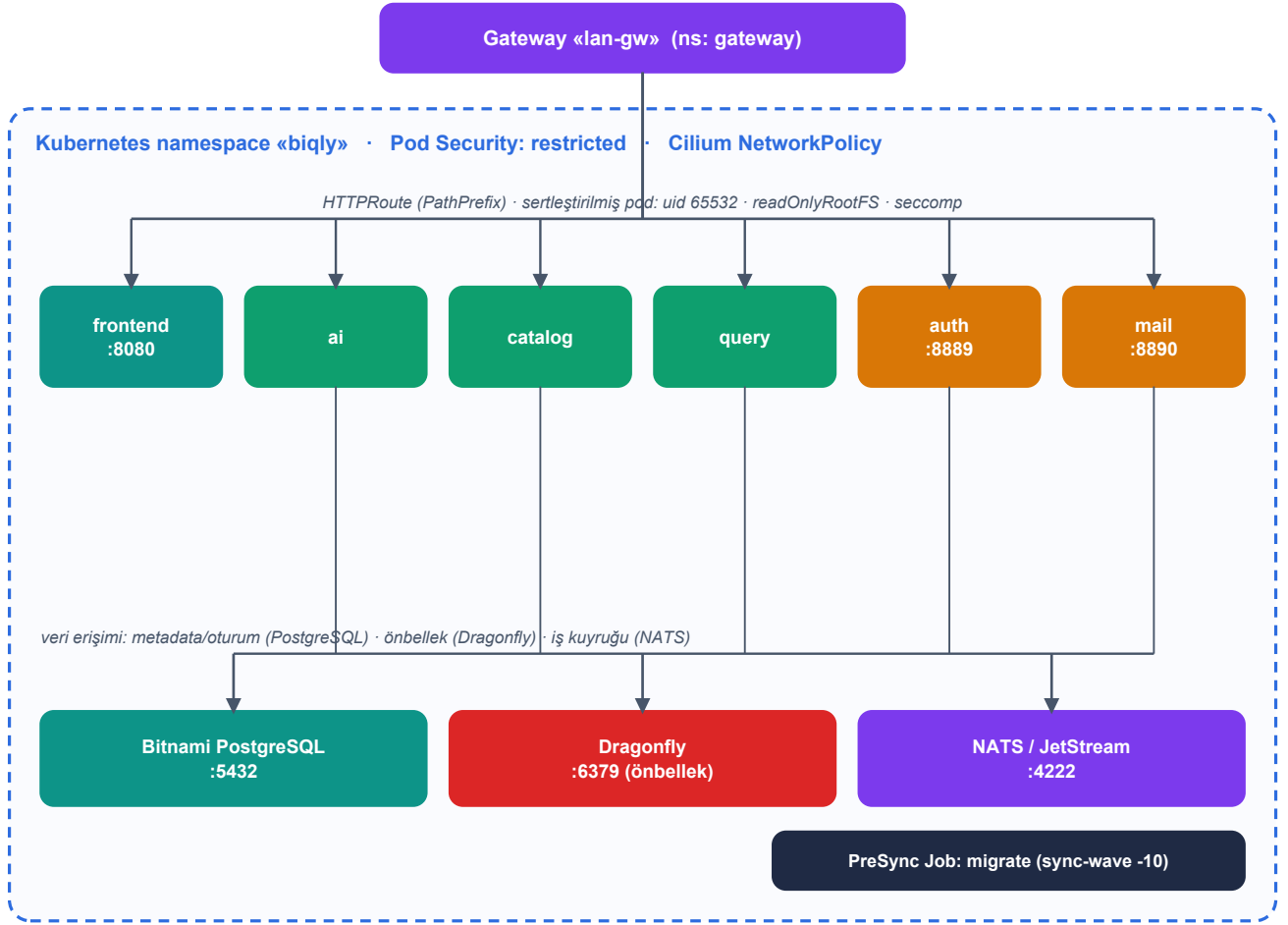
8.1 GitHub Actions İş Akışları

İş akışı	Amaç / Aşamalar
ci.yml	Ana hat. backend (vet → test -race+cov → AI eval regresyon kapısı → tüm cmd/* derle), lint (golangci v2.12.2), frontend (npm run check = eslint+prettier+knip+vitest+build), docker-api & docker-frontend push.
test.yml	Hafif kapı (PR'larda da): go test, golangci-lint, helm lint + template .
build-{ai,query,catalog,auth,mail,migrate}.yml	Servis başına Docker imaj derleme/push (QEMU+buildx, linux/arm64). Etiket: sha-<commit>. migrate her main push'ında derlenir.
semgrep.yml	SAST: p/security-audit, p/golang, p/react, p/typescript, p/owasp-top-ten → SARIF → fail-gate → GitHub Security. Push/PR + haftalık cron.

8.2 Docker İmajları

- **8 imaj**, hepsi çok-aşamalı, `linux/arm64`.
- **scratch tabanlı** (ai, query): yalnızca statik binary + CA paketi; `USER 65534`.
- **distroless tabanlı** (auth, mail): iki binary (servis + göç) + gömülü SQL göçleri; `nonroot`.
- **api**: `ARG SERVICE` ile çok-binary Dockerfile. **frontend**: Vite build → `nginx:alpine` (8080, SPA fallback + güvenlik başlıkları).

8.3 Kubernetes Runtime Topolojisi



Şekil 6 — «biqly» namespace'indeki çalışma-zamanı dağıtım topolojisi.

- **Helm umbrella chart:** namespace (Pod Security: restricted), Bitnami PostgreSQL, Dragonfly, NATS, Cilium NetworkPolicy'ler, gözlemlenebilirlik yığını (OTEL/Jaeger/Vector/Prometheus/Grafana/Alertmanager).
- **Sertleştirilmiş pod'lar:** runAsNonRoot, uid 65532, readOnlyRootFilesystem, tüm capability'ler düşürülür, seccomp RuntimeDefault, wait-for-postgres initContainer.
- **ArgoCD:** main dalından, values-prod.yaml ile otomatik senkron (prune + selfHeal); argocd-image-updater sha- etiketlerini git'e yazarak imajları otomatik yükseltir.
- **Göçler:** PreSync hook Job'ları (sync-wave ile sıralı); metadata göçü catalog imajının sha'sıyla kilit-adımda çalışır; ConfigMap migrations.biqly.io/revision anotasyonu senkron tetikler.

9. Best-Practice Değerlendirmesi

Bu bölüm yedi mühendislik boyutunu kanıta dayalı olarak değerlendirir. Her boyut için güçlü yönler, riskler ve somut öneriler verilir. İşaretler:

• **Güçlü** • **İzlenmeli** • **Risk**

9.1 Güvenlik

Değerlendirme: GÜÇLÜ

Güçlü yönler. Asimetrik **RS256 JWT** (issuer/audience doğrulamalı, MFA için ayrı audience); çift-gönderim CSRF + `subtle.ConstantTimeCompare` (zamanlama-güvenli); **mutlak + boşta** oturum süreleri, hash'li refresh token, cihaz parmak-izi; MFA/TOTP, WebAuthn, RBAC, LDAP, OAuth; sağlayıcı anahtarları **AES-256-GCM** ile şifreli; text-to-SQL enjeksiyon savunması **katmanlı** (read-only koruma + LogicalQuery whitelist doğrulaması + tam parametrelili yürütme + EXPLAIN dry-run); güvenilir-proxy IP doğrulaması; semgrep CI kapısı.

Riskler / boşluklar. HSTS varsayılan olarak **kapalı** (prod'da açmayı unutma riski); güvenlik-başlıkları middleware'inde **CSP / X-Frame-Options bulunamadı** (yalnız HSTS ailesi); read-only koruması SQL parser yerine anahtar-kelime eşleştirmesi kullanıyor (ama birincil savunma LogicalQuery doğrulaması).

Öneriler. (1) Middleware'e CSP + X-Frame-Options ekleyin. (2) Prod konfig doğrulamasında HSTS'i zorunlu (fail-closed) kılın.

9.2 Test

Değerlendirme: GENİŞ KAPSAM, DEĞİŞKEN DERİNLİK

Güçlü yönler. 200+ `_test.go` dosyası; yoğunluk tam da kritik yerlerde: handlers (18), ai (17), query (14), auth (11), semantic (10), security (8+4). Benchmark'lar mevcut; **race testi** zorunlu kapının parçası (`go test -race`); golden-file testleri; frontend 22 vitest dosyası; AI eval senaryoları (golden/ambiguity/repair + LLM judge).

Riskler / boşluklar. Birkaç üretim-yolu paketinde ince kapsam (her birinde 1 test): `datasource/{postgres,mysql,clickhouse,sqlserver}` (asıl DB sürücüleri — lehçe hatalarının saklandığı yer), dashboard, queue, config. Frontend test sayısı bileşen yüzeyine göre mütevazı.

Öneriler. (1) Her veri-kaynağı adaptörü için lehçeye özel entegrasyon testleri ekleyin. (2) Paket başına kapsam yüzdesini CI'da eşikle kapıya bağlayın (`coverage.out` zaten üretiliyor).

9.3 Kod Kalitesi & Mimari

Değerlendirme: GÜÇLÜ

Güçlü yönler. Temiz `cmd/internal/pkg` ayrımı; kararlılık gradyanı doğru (çekirdek paketler düşük instability, yaprak/orkestrasyon katmanları yüksek). Hata yönetimi Go 1.26 ev-kurallarına güçlü uyum: **143× errors.Is**, **12× errors.AsType**, **yalnızca 1 eski errors.As**, **0 ham == karşılaştırması**.

Riskler / boşluklar. Bir gerçek **tanrı-nesnesi**: `AIConfig` (45 alan, 13 metot, skor 84 — KRİTİK) — sağlayıcı/model DB-yönetimi göçü bu struct'ı aştı. Orkestratör `ai.Service.Process` en yüksek karmaşıklıkta (`//nolint:gocyclo,gocognit,funlen taşıyor`) — bilinçli ama bir bakım sıcak-noktası.

Öneriler. (1) `AIConfig`'i amaç-bazlı alt-konfiglere (query/embedding/translation) ayırın. (2) Öz-tutarlılık ve onarım dallarını adlandırılmış yardımcılara çıkararak nolint'i emekliye ayırın.

9.4 Gözlemlenebilirlik

Değerlendirme: KARIŞIK — TRACING BOŞLUĞU

Güçlü yönler. Yapılandırılmış loglama (`log/slog`), **Prometheus** metrikleri (35 alanlı kayıt), Helm'de tam yığın (OTEL collector, Jaeger, Vector, ServiceMonitor, Grafana, Alertmanager) ve **6 SLO-tarzı alarm kuralı** (hata-bütçesi yanması, servis başına gecikme); health/readiness uçları.

Riskler / boşluklar. **OTEL tracing koda enstrümente edilmemiş:** `otel` yalnızca dolaylı bağımlılık; `otel.Tracer` / `StartSpan` / `otelhttp` kullanımı yok. Yani Jaeger+collector dağıtılı olsa da uygulamalar span üretmiyor — LLM→derle→çalıştır yolunun dağıtık izlemesi pratikte yok. Bu, en büyük gözlemlenebilirlik boşluğudur ve **LLM gecikme** endişesi nedeniyle tam da en kritik yerde eksiktir.

Öneriler. (1) Her `cmd/*/main.go`'da `tracer` provider'ı bağlayın; `otelhttp` middleware + LLM/derleme/yürütme etrafında span ekleyin. (2) O zamana dek Jaeger'ı dokümanda 'hedef' olarak işaretleyin.

9.5 Performans

Değerlendirme: İYİ, ÖLÇÜME DAYALI

Güçlü yönler. Veri-kaynağı **bağlantı havuzu önbellesi** + `singleflight` tekilleştirme (thundering-herd önler, benchmark'lı); sürüm-farkında **sorgu fingerprint'i** (semantik model değişince önbellesi geçersiz kılar); AI yanıt + ambiguity önbellekleri; Dragonfly önbellek katmanı; CLAUDE.md'de ciddi sıcak-yol disiplini (`prealloc`, `strings.Builder`, `pprof`) ve golangci ile zorlanan `prealloc/perfsprint`; sıcak-yol JSON için `sonic/json-iterator`.

Riskler / boşluklar. **LLM gecikmesi** baskın maliyet; alarmı var ama tracing olmadan p99'u nedensel olarak atfetmek zor. Öz-tutarlılık oylaması (N aday) LLM tur sayısını ve maliyeti çoğaltır.

Öneriler. (1) Gecikmeyi (LLM vs derle vs DB) ayırtırmak için aşama-bazlı tracing ekleyin. (2) Öz-tutarlılık N'inin konfig-kapılı ve makul prod varsayılanlı olduğundan emin olun.

9.6 AI / LLM Mühendisliği

Değerlendirme: OLGUN — AÇIK FARKLILAŞTIRICI

Güçlü yönler. İki aşamalı mimari: LLM ham SQL değil *LogicalQuery* üretir; semantik modele karşı doğrulanır ve *sonra* lehçe SQL'e derlenir — bu hem doğru tasarım hem de enjeksiyon savunmasının temelidir. **Onarım döngüsü** (yapısal hata bağlamıyla yeniden prompt + kademeli bağlam katmanları), **EXPLAIN dry-run** doğrulama kapısı, **öz-tutarlılık oylaması**, **güven skorlaması**, **belirsizlik (ambiguity) işleme** ve gerçek bir **eval koşum hattı** (golden case + LLM judge + regresyon testi) ile `cmd/export-sft` ince-ayar geri-besleme döngüsü. DB'den yönetilen, AES-GCM şifreli sağlayıcı/model.

Riskler / boşluklar. Orkestratör; ambiguity + önbellek + oylama + onarımı tek çok-büyük fonksiyonda yoğunlaştırıyor (§9.3). Eval koşum hattı var ama CI'da kapı olarak çalıştığı doğrulanamadı — değilse SQL doğruluğundaki regresyonlar yayına çıkabilir.

Öneriler. (1) Eval/regresyon paketini (küçük bir golden alt-kümesi bile) CI kapısı yapın. (2) Orkestratörü ayrıştırın (§9.3 ile ortak).

9.7 DevX / Sürdürülebilirlik

Değerlendirme: MÜKEMMEL

Güçlü yönler. **golangci-lint v2** ~55 linter (gosec, errorlint, contextcheck, bodyclose, rowserrcheck, sqlclosecheck, noctx, sloglint, fmt.Print* ve panic'i yasaklayan forbidigo); frontend kapısı CI-eşdeğeri (ESLint + jsx-a11y + security, Prettier, **knip** ölü-kod tespiti, vitest, build); `deadcode -test` Go kapısında; zorunlu pre-commit denetimleri; gerçek i18n; 8 temiz `cmd/` giriş noktası; ADR dizini.

Riskler / boşluklar. ESLint `--max-warnings 1500` ile çalışıyor — regresyonları maskeleyebilen çok yüksek bir uyarı tavanı. Repo kökünde başıboş derleme çıktıları (`auth.test`, `app.test`, `workspace.test`, `coverage.out`) — `gitignore`'lanmalı.

Öneriler. (1) `--max-warnings` tavanını zamanla 0'a doğru kademeli düşürün. (2) Kökteki `*.test` binary'lerini ve `coverage.out`'u `.gitignore`'a ekleyin.

10. Sonuç ve Yol Haritası

Biqlı; disiplinli mimarisi (temiz cmd/internal/pkg, doğru kararlılık gradyanı, modern Go 1.26 hata deyimleri), katmanlı ve düşünülmüş güvenliği ve özellikle **standardın üzerindeki AI/text-to-SQL motoruyla** üretime hazır, geç-aşama bir kod tabanıdır. Onu “tam olgun”dan ayıran üç boşluk yereldir ve yeniden-yazım gerektirmeden kademeli olarak giderilebilir.

Önceliklendirilmiş öneriler

Öncelik	Aksiyon	Etki
Yüksek	OTEL tracing'i kodda enstrümante et (LLM/derle/yürüt span'leri)	Gecikme görünürlüğü; en yüksek getiri
Yüksek	AI eval/regresyon paketini CI kapısı yap	SQL doğruluk regresyonlarını önler
Orta	Veri-kaynağı sürücüleri için lehçe entegrasyon testleri	Sürücü/lehçe hatalarını yakalar
Orta	Güvenlik başlıklarına CSP + X-Frame-Options; prod'da HSTS zorunlu	Clickjacking/XSS sertleştirme
Orta	AIConfig'i ve Service.Process'i ayırıştır	Bakım borcunu azaltır
Düşük	ESLint uyarı tavanını kademeli düşür; *.test & coverage.out'u gitignore'la	Hijyen / regresyon görünürlüğü

Bu doküman, kaynak kod tabanının statik analizine (gograph), semgrep taramasına ve doğrudan kaynak incelemesine dayanmaktadır. Skor kartındaki sayısal değerler, kanıta dayalı niteliksel değerlendirmenin özetidir.